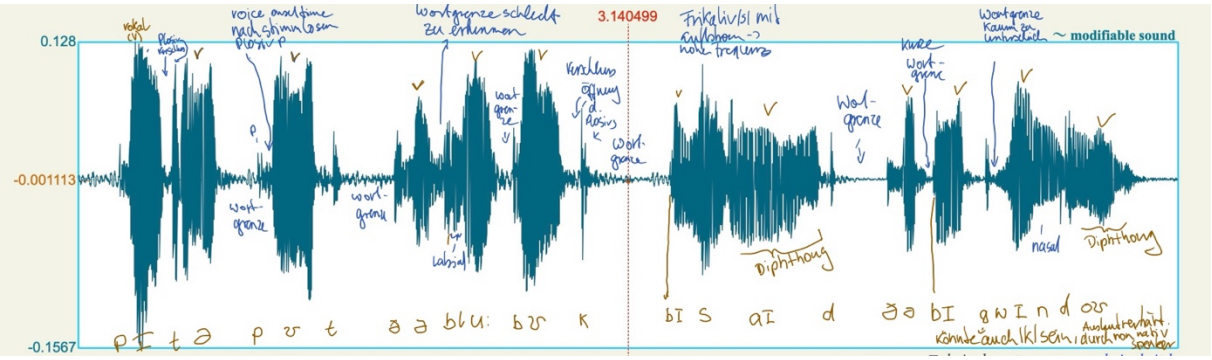
Task 1

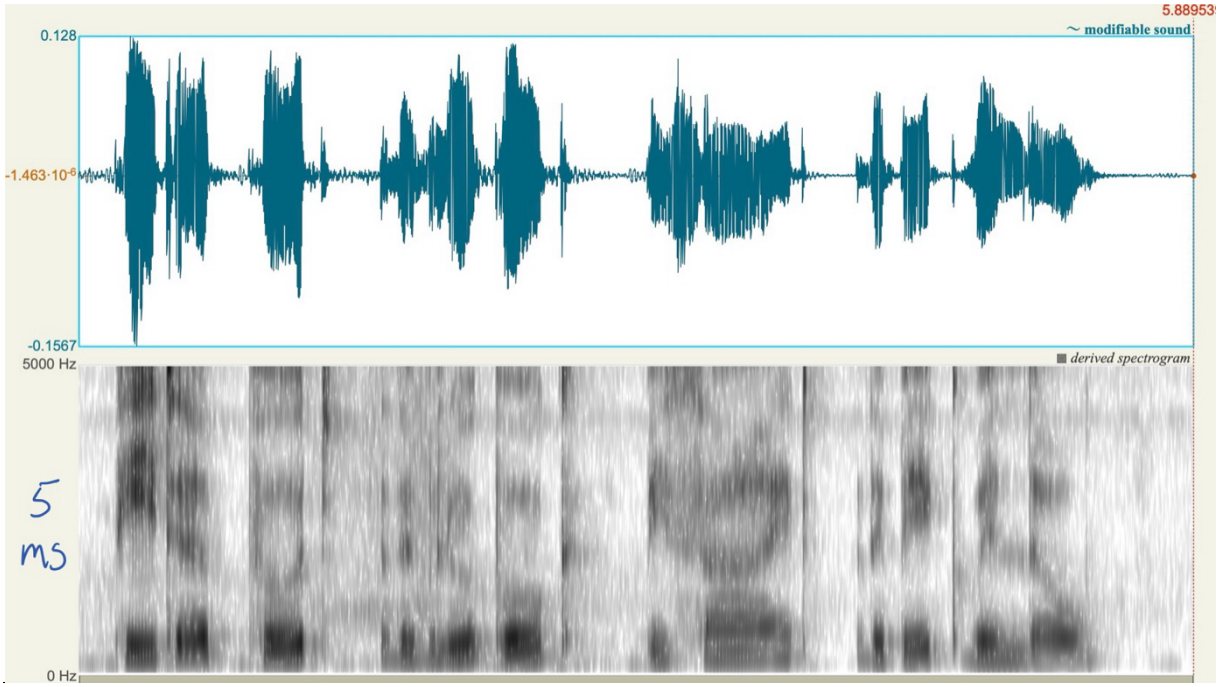
a) Praat, waveforms and spectrograms

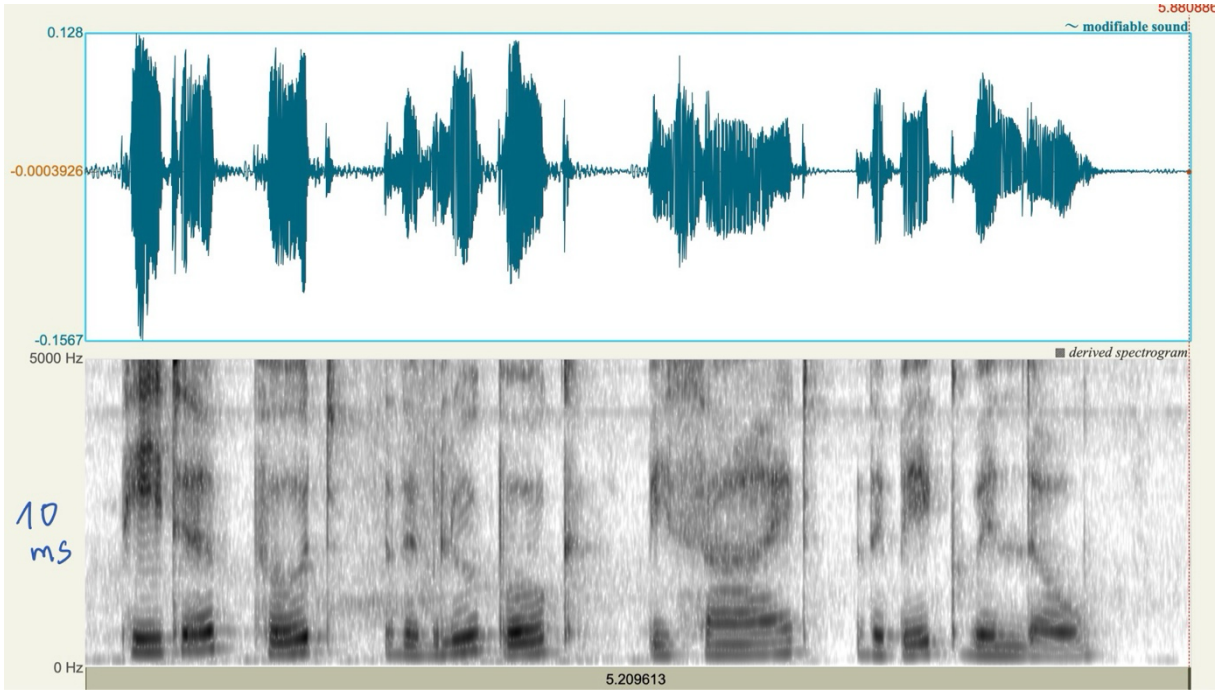
Examining the waveform of the sentence " Peter put the blue book beside the big window":

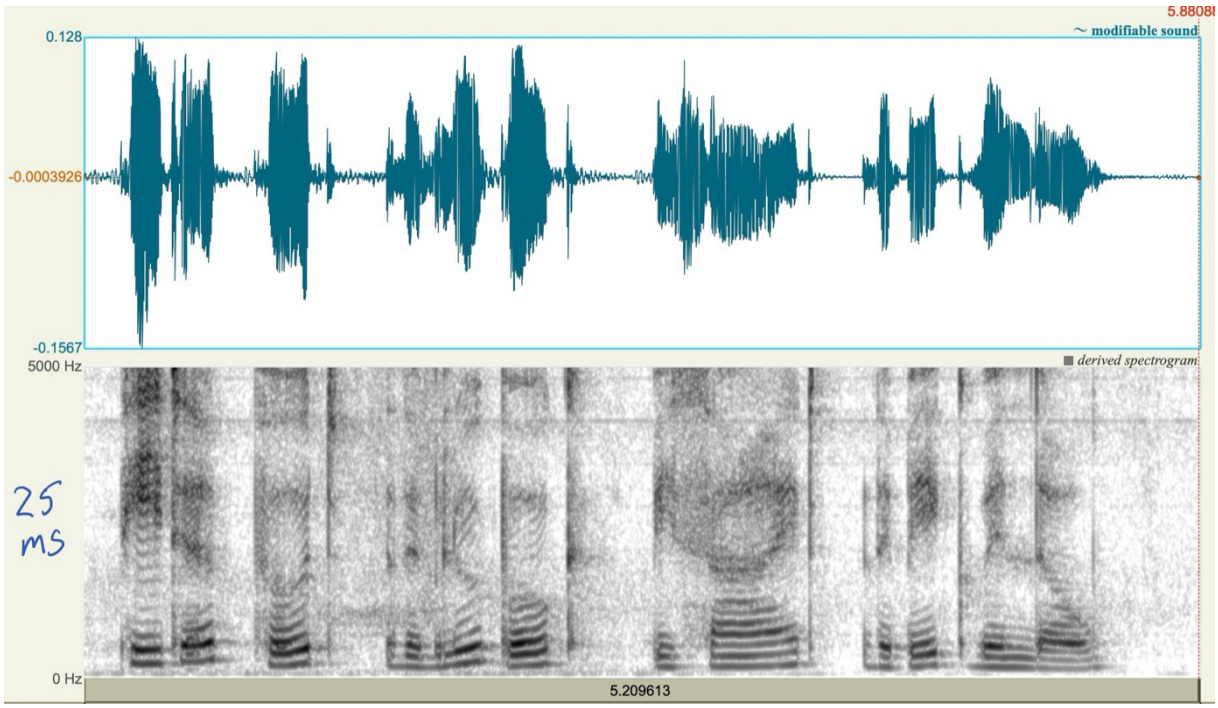


- Vowels are relatively easy to detect, although they are sometimes hard to differentiate from surrounding phonemes as in "blue" – the l and u - or "window" - ndow.
- The *bi* in *big* or *beside* is also hard to differentiate just visually.
- Word boundaries (very low frequency over a time period) are sometimes simple to detect (e.g., book _ beside):
- Sometimes they are less easy to differentiate (e.g., the _ big).
- Sometimes the voice onset time in a word after a voiceless plosive is easier to detect than a word boundary.
- It is very hard to describe the sentence based on the waveform alone.
  -> We tried to transcribe the sentence, respectively tried to match the parts of the waveforms to the phonemes, by using the audio signal and the visual waveform representation.

Spectrograms 5, 10, 25 ms:

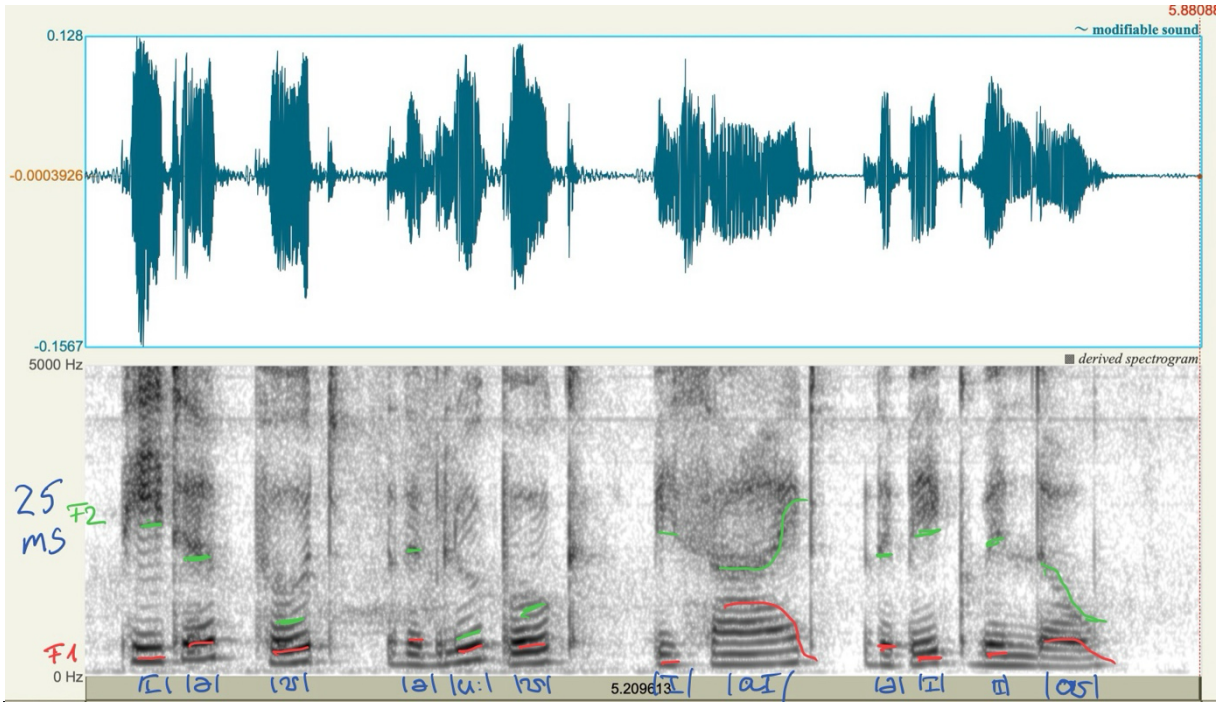

5.889535
~ modifiable sound
0.128
-1.463·10⁻⁶
-0.1567
derived spectrogram
5000 Hz
5 ms
0 Hz


~ modifiable sound
0.128
-0.0003926
-0.1567
derived spectrogram
5000 Hz
10 ms
0 Hz
5.209613

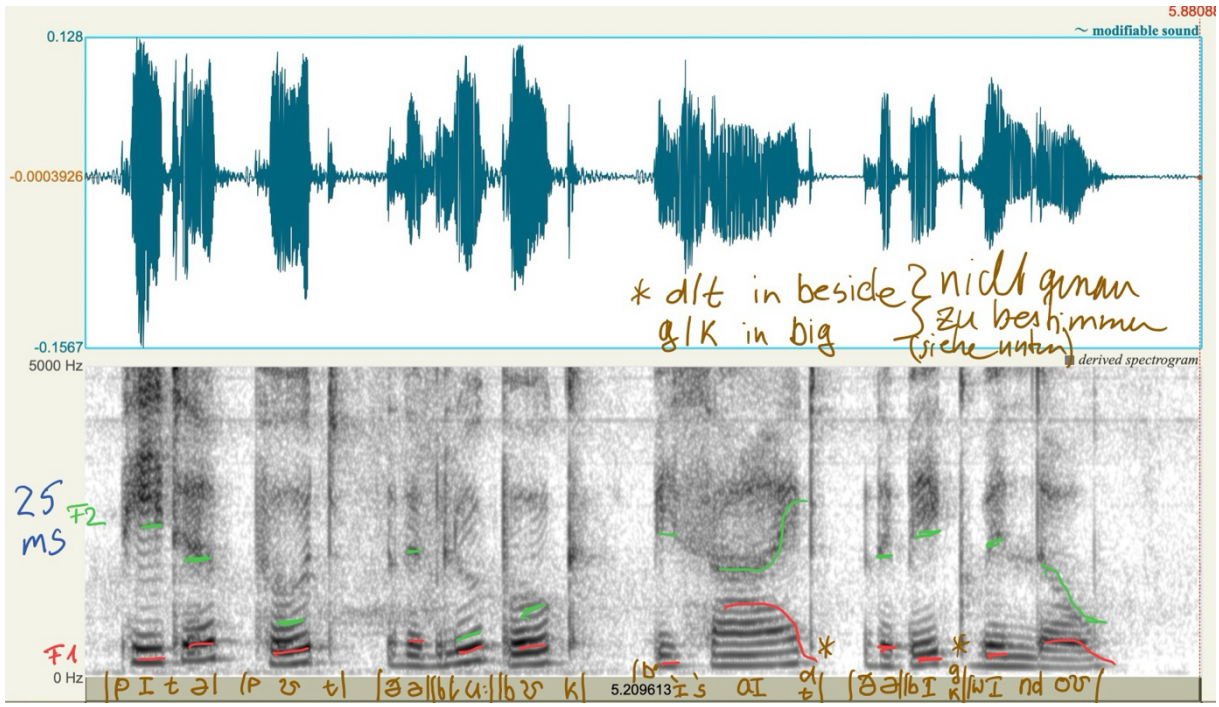


- Within the 25 ms window, the individual frequencies (harmonics) (black rings/stripes) are much more clearly visible than in the 5 ms window. This is because the 25 ms window is long enough to detect the full cycle of the tone while the 5 ms window can only detect a short uncomplete part. In the 5 ms window, the rings tend to merge with one another and are only visible as a blur. A 10 ms window fits somewhere in between.
- The 25 ms window is the best to examine the vowels. The individual frequencies can be seen more clearly allowing for optimal vowel detection based on their formants.
- In the 5 ms window you can instead see fine vertical lines. Each vertical line shows the vibration of the vocal folds – specifically the moment they close. This can be seen because of the very short window length of the spectrogram and therefore a high temporal resolution which useful to detect consonants. For example plosives are recognizable by a steady black vertical line which represents the burst of the plosive.
- So the best setting for speech analysis depends on what you want to analyze. We would use a 25ms window to analyze the vowels and a 5 ms window to analyse and detect consonants likes plosives with closures, bursts and temporal transitions.

Vowel Formants



Whole transcription



- The vowel "i" looks similar over different words, especially in *Peter*, *big* and *window*. The formants for the "i" are also similar in *beside* but the duration is a bit shorter (smaller black rings).
- The short *u* in *put* and *book* also looks similar in both words, as well as the "schwa" sound "e" in *Peter* and the word *the*.

- It is relatively easy to detect the plosives by the black vertical line but it is hard to differentiate which plosiv it actual is
- In the given spectrograms it is also hard to determine if a plosive is voiced or voiceless.
    - E.g. *big* should be pronounced with a voiced "g" at the end but as it is spoken by a native German speaker it could also be an unvoiced "g" pronounced as "k") because of the "Auslautverhärtung" in German (same with the "d" in beside). In the audio it sounds more like a "k".
- The diphthongs are both relatively long in their duration and they have a characteristic falling down or going up of the formants in the middle (like a curve)
- The fricative "*s"* in *beside* is also easy to detect because of its longer duration and its high frequency noise created by the airflow

b) Speech recognition failure

The speech recognition system was most likely trained exclusively on standard English (Oxford).

Dialects differ from standard English in their use of phonemes, rythm and emphasis. As a result, the meaning of spoken words changes or is lost compared to the standard English used in training.

Hypothesis: Although the characters recognized the problem, they did not use the pronunciation (standard English) with which the speech recognition system was most likely trained. This is why they are unable to give the speech recognition system an executable command (eleven).

The satirical sketch leaves it up to the viewer to determine whether the cause of this inability is a psychological or linguistic language barrier.

An Analytical Comparison of Dialect and Standard English

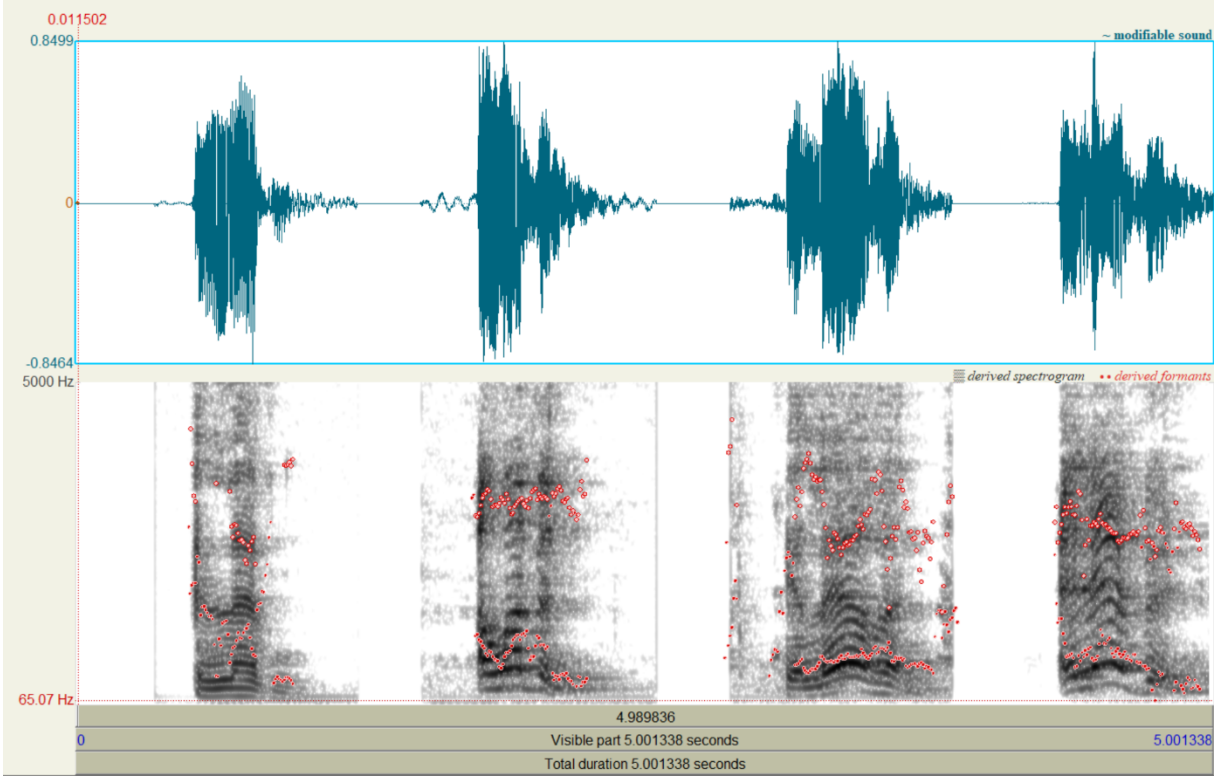


Fig. 1

1. Oxford: https://www.oed.com/dictionary/eleven_adj?tl=true&tab=pronunciation
2. Scottish: https://www.youtube.com/watch?v=MNuFcIRlwdc, ~ 29 s
3. „American“ aus https://www.youtube.com/watch?v=MNuFcIRlwdc , ~ 53 s
4. „English“ aus https://www.youtube.com/watch?v=MNuFcIRlwdc, ~ 01 m 14 s

In Fig. 1, the word “eleven” is spoken four times. The first sound wave (from the left) is standard English. To the right of it are three attempts by the characters to convey “eleven” as a command to the elevator’s speech recognition system. Both the contour (amplitude) of the sound waves over time (stress) and the arrangement of the formants F1/F2—shown as red markers in the spectrogram (vowels)—document the significant deviation in the characters’ pronunciation from standard English.

To experimentally verify the differences identified in the analysis, the speech recognition system could be tested using additional English dialects—a) Geordie (Newcastle), b) Scouse (Liverpool), and c) Yorkshire—with a defined number of different words, and a recognition rate could be determined. This would demonstrate whether and to what extent recognition depends on the individual’s pronunciation or correlates with the regional dialect.

The speech recognition system would also need to be trained on English dialects to expand the available vocabulary and improve recognition.

assignment-01\task-2-programming.py

```python
# Task 2 - Programming

# Task 2.1 Unterschied CMU Pronouncing Dictionary versus IPS:
# Bei dem CMU Pronouncing Dictionary http://www.speech.cs.cmu.edu/cgi-bin/cmudict
# unterscheidet sich die Codierung der Phoneme im wesentlichen dadurch von dem
# IPA-System, dass Großbuchstaben des lateinischen Alphabets verwendet werden.
# Bei dem IPA-System wird für jedes Phonem ein eigenes Zeichen genutzt
# (Bsp. ʃ für sch). Das CMU Pronouncing Dictionary ist hauptsächlich für das Englische
# ausgelegt, während das IPA-System international für alle Sprachen genutzt werden
# kann. Das Phonemen-Set im CMU besteht aus 39 Phonemen (des Englischen), aus denen
# die Worte zusammengesetzt werden. Betonungen werden durch die Indizes 0=Ohne Betonung,
# 1=primäre Betonung, 2=sekundäre Betonung abgebildet. Das CMU Pronouncing Dictionary
# nutzt das ARPABET https://en.wikipedia.org/wiki/ARPABET, ein Set phonetischer
# Transkriptions-Codes, die in den 1970er Jahren von der Advanced Research Projects
# Agency (ARPA) entwickelt wurde.
# Bsp: CMU big - B IH1 G, IPA /bɪg/

# Task 2.2 Load pronouncing library
import pronouncing

# Task 2.3 Looking up pronounciation of "Peter put the blue book ..."
sentence = ["Peter", "put", "the", "blue", "book", "beside", "the", "big", "window"]
for word in sentence:
    print(f"{word}: {pronouncing.phones_for_word(word.lower())}") # library needs lower()
print()

# Task 2.4 Write a function that returns True for slant or exact rhymes.
# The Chaos was taken from: https://ncf.idallen.com/english.html
# Erledigt: Write a function are_rhymes(word1, word2) that returns
# True for slant or exact rhymes (e.g., “skate” and “eight”) despite mismatched spellings.
# Prototyp läuft: Extract the nucleus and coda, zeigt True und False für Reimpaare an
# Festlegung "slant", siehe is_slant_rhyme()

def are_rhymes(word1, word2):
    """word1 und word2 sind Reime, wenn sie entweder ein exakter oder ein slant rhyme sind"""
    # vergleicht ob word1 in extrahierter Reimliste von word2 ist
    exact_rhyme = word1 in pronouncing.rhymes(word2)
    slant_rhyme = is_slant_rhyme(word1, word2)
    return exact_rhyme or slant_rhyme

def get_nucleus_and_coda(word: str) -> tuple[str, list[str]]:
    # Beispiel: beside
    # phones: 'B IH0 S AY1 D'
    # rhyming_part: AY1 D
    # nucleus: AY1
    # coda: D
    phones = pronouncing.phones_for_word(word)
    # um später Ergebnis True/False zu verstehen
    print(phones)

    if not phones:
```

```
        return None, None

    # Erste Aussprachevariante verwenden
    rhyming_part = pronouncing.rhyming_part(phones[0]).split()

    if not rhyming_part:
        return None, None

    # Ersten Laut als Nucleus, restliche Laute als Coda
    nucleus = rhyming_part[0]
    coda = rhyming_part[1:]

    return nucleus, coda


def is_slant_rhyme(word1, word2):
    # unsere Regel für slant Reim: gleicher Nucleus und unterschiedliche Coda
    # oder unterschiedlicher Nukleus und gleiche Coda
    nucleus1, coda1 = get_nucleus_and_coda(word1)
    nucleus2, coda2 = get_nucleus_and_coda(word2)

    # Wenn eines der Wörter nicht gefunden wurde
    if nucleus1 is None or nucleus2 is None:
        return False

    return ((nucleus1 == nucleus2 and coda1 != coda2) or
            (nucleus1 != nucleus2 and coda1 == coda2))

word1 = ""                                          # Ablage wort1 initiieren
word2 = ""                                          # Ablage wort2 initiieren
# eigenen lokalen Pfad in input_file eintragen oder hier direkt im NLPROC Ordner
input_file = "assignment-01\\the-chaos.txt"
with open(input_file, "r", encoding="utf-8") as file:
    for line in file:
        if len(line) > 1:                           # Leerzeilen überspringen
            end_word = line.split()[-1]             # letztes Wort holen
            end_word = end_word.strip(".,;:-?!\"'()*")  # entfernt Zeichen links und
rechts
            if word1 == "":                         # letztes Wort Satz 1 ablegen
                word1 = end_word
            elif word2 == "":                       # letztes Wort Satz2 ablegen
                word2 = end_word
                # abgelegtes Wortpaar auf Reim mit Funktion prüfen
                print(word1, word2, are_rhymes(word1.lower(), word2.lower()))
                print()
                word1 = ""                          # Ablage wort1 leeren
                word2 = ""                          # Ablage wort2 leeren

# Bsp.: test für leere Liste bei tortious, Wort gibt es in CMU nicht, daher False
# print(pronouncing.phones_for_word("tortious"))
# print(pronouncing.search(".*tortious.*"))

```

Output for poem "Chaos":

['K R IY0 EY1 SH AH0 N']
['P R OW0 N AH2 N S IY0 EY1 SH AH0 N', 'P R AH0 N AH2 N S IY0 EY1 SH AH0 N']
creation pronunciation True

['V ER1 S']
['W ER1 S']
verse worse True

['B IH1 Z IY0']
['D IH1 Z IY0']
busy dizzy True

['T EH1 R', 'T IH1 R']
['P R EH1 R', 'P R EY1 ER0']
tear prayer True

['P OW1 AH0 T']
['IH1 T', 'IH0 T']
poet it False

['HH ER1 D']
['W ER1 D']
heard word True

['B R IH1 T AH0 N']
['R IH1 T AH0 N']
Britain written True

['B EY1 D']
['P L AE1 D']
bade plaid True

['Y UW1']
['EY1 G Y UW2']
you ague True

['S P IY1 K']
[]
speak False

['V AY1 AH0', 'V IY1 AH0']
['K W AY1 ER0']
via choir True

['L OW1']
['T OW1']
low toe True

['T R IH1 K ER0 IY0']
[]
trickery Terpsichore False

['AY1 L Z', 'AY1 AH0 L Z']

[]
aisles reviles False

['S AY1 N IH0 NG']
['M AY1 N IH0 NG']
signing mining True

['S IH0 G AA1 R']
['F AA1 R']
cigar far True

['AE0 D M AY1 R']
['B R AY1 ER0']
admire brier True

['N OW1 N']
['T OW1 N']
known tone True

['B AE0 L M AO1 R AH0 L']
['L AO1 R AH0 L']
Balmoral laurel True

['W AY1 N D', 'W IH1 N D']
['M AE1 N K AY1 N D']
wind mankind True

[]
['HH EH1 DH ER0']
chamois-leather heather False

['L AE1 B ER0 IH2 N TH']
['P L IH1 N TH']
labyrinth plinth True

[]
['S EH1 V ER0 D']
endeavoured severed False

['S OW1 L']
['P AH0 T R OW1 L']
soul patrol True

['B AE0 L EY1']
['SH AE1 L EY2', 'SH AH0 L EY1']
ballet chalet True

['F UW1 D']
['W UH1 D']
food would True

['P AA0 R K EY1']
['K AA1 K IY0', 'K AE1 K IY0']
parquet khaki False

['B R AO1 D']

['R IH0 W AO1 R D', 'R IY0 W AO1 R D']
broad reward True

['K R OW0 K EY1']
['OW1 K EY1']
croquet OK True

['S IH1 V']
['L AY1 V', 'L IH1 V']
sieve live True

['HH AY1 ER0']
['TH EY1 L Y AH0']
higher Thalia False

['HH UW1 T']
['M IH1 N AH0 T', 'M AY0 N UW1 T', 'M AY0 N Y UW1 T']
hoot minute True

['P R IY0 S IH1 ZH AH0 N']
['T R AE0 N Z IH1 SH AH0 N']
precision transition True

['R AY1 M']
['P EH1 R AH0 D AY2 M']
rhyme paradigm True

['IY1 Z IY0']
['G R IY1 S IY0']
easy greasy True

['R IH0 V AY1 Z', 'R IY0 V AY1 Z']
['L AH1 L AH0 B AY2 Z']
revise lullabies True

['N AO1 SH AH0 S']
[]
nauseous tortious False

['HH OW1 P']
['EH1 N V AH0 L OW2 P']
hope envelope True

['IH1 T', 'IH0 T']
[]
it davit False

['SH IY1 K']
['EY1 K']
Sheik ache True

['HH EH1 V AH0 N']
['IH0 L EH1 V AH0 N', 'IY1 L EH0 V AH0 N']
heaven eleven True

['AH0 L AW1 D']

['V AW1 D']
allowed vowed True

['M AO0 R OW1 V ER0']
['D OW1 V ER0']
moreover Dover True

['P R IH0 S AY1 S', 'P R IY0 S AY1 S']
['L AY1 S']
precise lice True

['AH0 N S T EY1 B AH0 L']
['L EY1 B AH0 L']
unstable label True

['K AH0 N AE1 L']
['P AE1 L']
canal pal True

['K AA1 N D UW0 IH0 T', 'K AA1 N JH UW0 IH0 T', 'K AA1 N D W IH0 T']
['IH1 T', 'IH0 T']
conduit it False

['T EH1 L']
['M AO1 L']
tell Mall True

['AY1 ER0 N']
['L AY1 AH0 N']
iron lion True

['CH EH1 R']
['M EY1 ER0']
chair mayor False

['K L AE1 M ER0']
['HH AE1 M ER0']
clamour hammer True

['P AH0 Z EH1 S']
['AE1 D R EH2 S', 'AH0 D R EH1 S']
possess address True

['L UW0 T EH1 N AH0 N T S']
['P EH1 N AH0 N T S']
lieutenants pennants True

['K OW1 M']
['HH OW1 M']
comb home True

['TH IH1 K ER0']
['L IH1 K ER0']
thicker liquor True

['T R UW1']

['AH0 D UW1']
true ado True

['AE1 NG G ER0']
[]
anger clangour False

['AE1 N T', 'AO1 N T']
['G R AE1 N T']
aunt grant True

['S IY1 N IH0 K']
[]
scenic hygienic False

['K L OW1 S', 'K L OW1 Z']
['D OW1 S']
close dose True

[]
['IY1 G AH0 L']
inveigle eagle False

['M IY1 N']
['M AE1 G AH0 Z IY2 N']
mean magazine True

['P EH1 N IY0']
['M EH1 N IY0']
penny many True

['P IH1 R']
['T IY1 R']
pier tier True

['EH1 R IH0 NG']
['S T ER1 IH0 NG']
erring stirring False

['T R OW1 V']
['K OW1 V']
trove cove True

['R AY1 B AA0 L D']
['N IH1 B AH0 L D']
Ribald nibbled False

['N AO1']
['P SH AW1']
gnaw pshaw True

['IH1 T', 'IH0 T']
['B AH1 F AH0 T', 'B AH0 F EY1']
it buffet False

['B UW1 N']

['IH2 M P Y UW1 N']
boon impugn True

['S T ER1 L IH0 NG']
['Y ER1 L IH0 NG', 'Y IH1 L IH0 NG']
sterling yearling True

[]
['HH IH1 N T']
mezzotint hint False

['AH0 T EH1 N SH AH0 N']
['M EH1 N SH AH0 N']
attention mention True

['P AO1 Z']
['Y UH1 R Z', 'Y AO1 R Z', 'Y ER0 Z']
paws yours False

['IH2 N K L UW1 D AH0 D', 'IH2 N K L UW1 D IH0 D']
['D IH1 D', 'D IH0 D']
included did False

['Y UW1 N IH0 K AO2 R N']
['S T R AE1 CH AH0 N']
unicorn Strachan False

['K AH1 M L IY0']
[]
comely Cholmondeley False

['P R AW1 D']
['M AH0 K L AW1 D']
proud McLeod True

['V AY1 AH0 L']
['D IH0 N AY1 AH0 L']
vial denial True

['R IY1 M']
['S K IY1 M']
ream scheme True

['SH UH1 R L IY0']
['R AO1 L IY0']
Surely Raleigh False

['S EY1']
['S OW1 B R AH0 K EY2', 'S AH2 B R AH0 K EH1 T']
say sobriquet True

['IH1 N V AH0 L AH0 D', 'IH1 N V AH0 L IH0 D', 'IH2 N V AE1 L AH0 D']
['P AE1 L AH0 D']
invalid pallid True

['L UH1 K T']

['B UH1 K T']
looked booked True

['AE2 F R AH0 D AY1 T IY0']
['F L AY1 T IY0']
Aphrodite flighty True

['AE2 N T IH0 P OW2 D IY0 Z']
[]
antipodes obsequies False

['G AY1 Z ER0']
['R EY1 Z ER0']
geyser razor True

['P Y UH1 R']
['M AH0 CH UH1 R', 'M AH0 T Y UH1 R']
pure mature True

['G L AH1 M L IY0']
[]
glumly dumbly False

['F AE1 N']
['AA1 R T AH0 Z AH0 N']
fan artisan False

['Y UW1']
['D AH1 B AH0 L Y UW0']
you w False

['JH EH1 M Z']
['T EH1 M Z']
gems Thames True

[]
['EH1 M']
Streatham em False

['AE1 N TH AH0 N IY0']
['AE0 NG Z AY1 AH0 T IY0']
Anthony anxiety False

['AO0 L B IY1 IH0 T']
['IH1 T', 'IH0 T']
albeit it False

['V OY1 S']
['CH OY1 S']
voice choice True

['F IH1 NG G ER0']
['L IH1 NG G ER0', 'L IH1 NG ER0']
finger linger True

['G EY1 JH']

['EY1 JH']
gauge age True

['V EH1 R IY0']
['B EH1 R IY0']
very bury True

['L AA1 TH']
['OW1 TH']
loth oath True

[]
[]
oppugners banjo-tuners False

['K AH0 N UW1 Z']
['Y UW1 S', 'Y UW1 Z']
canoes use True

['L IH1 T AH0 L']
[]
little victual False

['HH AY1 T']
['Y UW1 N AY2 T']
height unite True

[]
['HH AY1 F ER0', 'HH EH1 F ER0']
deafer heifer False

['L EY1 T']
['S IH0 D EY1 T']
late sedate True

['P AH0 S IH1 F IH0 K']
['S AY2 AH0 N T IH1 F IH0 K']
pacific scientific True

['F AO1 R']
['AA1 R K AH0 N S AA2']
four Arkansas False

['V AA1 M AH0 T']
['IH1 T', 'IH0 T']
vomit it False

['AE1 L AH0 B AY2']
['ER0 AY1']
alibi awry True

['EH1 R IY0 AH0']
['M AH0 L EH1 R IY0 AH0']
area malaria True

['K L IY1 N']

['M ER0 IY1 N']
clean marine True

['IH2 T AE1 L Y AH0 N']
['B AH0 T AE1 L Y AH0 N']
Italian battalion True

['Y IY1', 'Y EH1']
['K IY1', 'K EY1']
ye quay True

['F IY1 V ER0']
['R AH0 S IY1 V ER0', 'R IH0 S IY1 V ER0', 'R IY0 S IY1 V ER0']
fever receiver True

['S EY1 F']
['R AA1 L F']
safe Ralf False

['K AH0 N EH1 R IY0']
['EH1 R IY0']
canary eyrie True

['G R IH1 M AH0 S']
['B AE1 S', 'B EY1 S']
grimace bass False

['V ER1 JH IH0 NG']
['S K ER1 JH IH0 NG']
verging scourging True

['T EH1 R', 'T IH1 R']
['EH1 R']
tear heir True

['AO1 F AH0 N', 'AO1 F T AH0 N']
['AO1 R F AH0 N']
often orphan True

['S AO1 S']
['K R AO1 S']
sauce cross True

['P AH1 T IH0 NG', 'P UH1 T IH0 NG']
['SH AH1 T IH0 NG']
Putting shutting True

['R IH0 Z EH1 N T', 'R IY0 Z EH1 N T']
['P AA1 R L AH0 M AH0 N T']
resent Parliament False

['IY1 V IH0 N']
['S T IY1 V AH0 N']
even Stephen True

['JH ER1 K']

['W ER1 K']
jerk work True

[]
['N UW1 Z P EY2 P ER0']
vapour newspaper False

['JH IH1 S T']
['G R IH1 S T']
gist grist True

['D AY1 V ER0 Z']
[]
divers fivers False

['R OW1 L']
['P OW1 L']
roll poll True

['S AY1 K IY0']
['S P AY1 K IY0']
Psyche spiky True

['W IH1 T S']
['G R IH1 T S']
wits grits True

['T AH1 N AH0 L']
[]
tunnel gunwale False

['W AY1 T']
['IH2 N D AY1 T']
Wight indict True

['R AE1 DH ER0', 'R AH1 DH ER0']
['F AA1 DH ER0']
rather father True

['IH0 N AH1 F', 'IY0 N AH1 F']
['T AH1 F']
enough tough True

['S AH1 P']
['AH1 P']
sup UP True